Transactional Outbox

Outbox · Atomically save order + outbox row, relay later · Build with Waqar

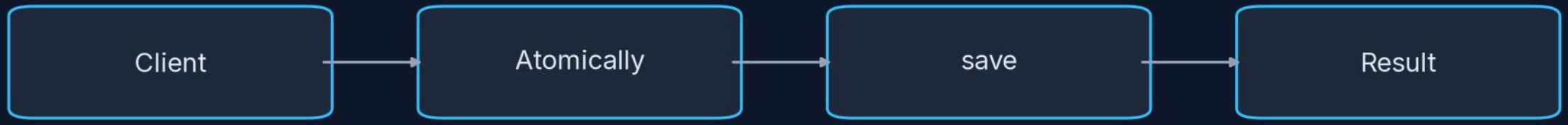
Today's Hook

- Today's force: Atomically save order + outbox row, relay later
- We earn Outbox by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: OutboxRow, OutboxStore, OutboxRelay

The Problem

- Scenario: Atomically save order + outbox row, relay later
- Constraints: solve «Atomically save order + outbox row, relay later» with Outbox — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Atomically save order + outbox row, relay later
- Success looks like: saved o1

Structure



Outbox: Atomically save order + outbox row, relay later

Real-World Scenario

- Running example: Atomically save order + outbox row, relay later
- Key types: OutboxRow, OutboxStore, OutboxRelay
- Happy-path stdout cue: saved o1
- Failure-path stdout cue: caught=broker down

Naive Code

```java
package com.buildwithwaqar.patterns.enterprise.outbox.problem;

import java.util.*;

public final class NaiveOrderPub {
  public static void place(String id){ /* save order */ System.out.println("saved "+id); /* publish may fail after */ throw new RuntimeException("broker down"); }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Atomically save order + outbox row, relay later
- Without Outbox, call sites absorb the wrong details.
- Failure mode to show on camera: caught=broker down
- New requirements force shotgun edits across the naive code.

Solution

```java
package com.buildwithwaqar.patterns.enterprise.outbox.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

record OutboxRow(String id, String payload, boolean published){}
final class OutboxStore {
  private final List<OutboxRow> rows=new ArrayList<>();
  private final List<String> orders=new ArrayList<>();
  void placeOrder(String id, String payload){ orders.add(id); rows.add(new OutboxRow(id, payload, false)); }
  List<OutboxRow> unpublished(){ return rows.stream().filter(r->!r.published()).toList(); }
  void markPublished(String id){
    for (int i=0;i<rows.size();i++) if(rows.get(i).id().equals(id)) rows.set(i, new OutboxRow(id, rows.get(i).payload(), true));
  }
  List<String> orders(){ return List.copyOf(orders); }
}
final class OutboxRelay {
  private final OutboxStore store; private final List<String> published=new ArrayList<>();
  OutboxRelay(OutboxStore store){ this.store=store; }
  int relayOnce(){ int n=0; for(var r: store.unpublished()){ published.add(r.payload()); store.markPublished(r.id()); n++; } return n; }
  List<String> published(){ return List.copyOf(published); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.enterprise.outbox.problem;
import java.util.*; import java.util.concurrent.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Outbox: order + event atomically ===");
    System.out.println("--- naive ---");
try { NaiveOrderPub.place("o1"); } catch(Exception e){ System.out.println("caught="+e.getMessage()); }
System.out.println("--- outbox ---");
var store=new OutboxStore(); store.placeOrder("o1","OrderPlaced:o1");
var relay=new OutboxRelay(store); System.out.println("relayed="+relay.relayOnce()); System.out.println("published="+relay.published());
System.out.println("relayedAgain="+relay.relayOnce());
  }
}
```

Walkthrough Output

```
=== Outbox: order + event atomically ===
--- naive ---
saved o1
caught=broker down
--- outbox ---
relayed=1
published=[OrderPlaced:o1]
relayedAgain=0
```

Step-by-Step Trace

- Match stdout to stages — look for: saved o1
- Then the injected failure: caught=broker down
- Re-run: java -cp ... 34-outbox ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
package com.buildwithwaqar.patterns.enterprise.outbox.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

record OutboxRow(String id, String payload, boolean published){}
final class OutboxStore {
  private final List<OutboxRow> rows=new ArrayList<>();
  private final List<String> orders=new ArrayList<>();
  void placeOrder(String id, String payload){ orders.add(id); rows.add(new OutboxRow(id, payload, false)); }
  List<OutboxRow> unpublished(){ return rows.stream().filter(r->!r.published()).toList(); }
  void markPublished(String id){
    for (int i=0;i<rows.size();i++) if(rows.get(i).id().equals(id)) rows.set(i, new OutboxRow(id, rows.get(i).payload(), true));
  }
  List<String> orders(){ return List.copyOf(orders); }
}
final class OutboxRelay {
  private final OutboxStore store; private final List<String> published=new ArrayList<>();
  OutboxRelay(OutboxStore store){ this.store=store; }
  int relayOnce(){ int n=0; for(var r: store.unpublished()){ published.add(r.payload()); store.markPublished(r.id()); n++; } return n; }
  List<String> published(){ return List.copyOf
```

Trade-offs

- Use Outbox when «Atomically save order + outbox row, relay later» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Outbox in product language.

Related Patterns

- Name the pattern usually confused with Outbox and separate the forces.
- Sketch the roles for «Atomically save order + outbox row, relay later» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Atomically save order + outbox row, relay later
- Naive pain → Outbox boundary → Demo stdout.
- Remember the failure cue: caught=broker down
- Break a ProblemTest assertion, watch it fail, restore it.